API Gateway

Smart Sales Store | module `api-gateway` | Java 21, Spring Boot 3.3.4, Spring Cloud Gateway 2023.0.3

1. Overview

The API Gateway is the single entry point to the Smart Sales Store backend. Clients such as the React frontend never call a microservice directly: every request goes to the gateway on port **8080**, which authenticates it, picks the right service from the URL path, and protects the system from failing downstream services. It is built on Spring Cloud Gateway (reactive, non-blocking) and runs as its own Gradle module and Docker image.

Responsibilities

- **Routing** of `/api/**` paths to the auth, user, customers and products services.
- **Authentication**: validates JWT bearer tokens and passes the caller's identity downstream.
- **Resilience**: circuit breakers with fallback responses, plus retries on the auth route.
- **Cross-cutting concerns**: CORS policy, Actuator health and route inspection endpoints.

2. Architecture

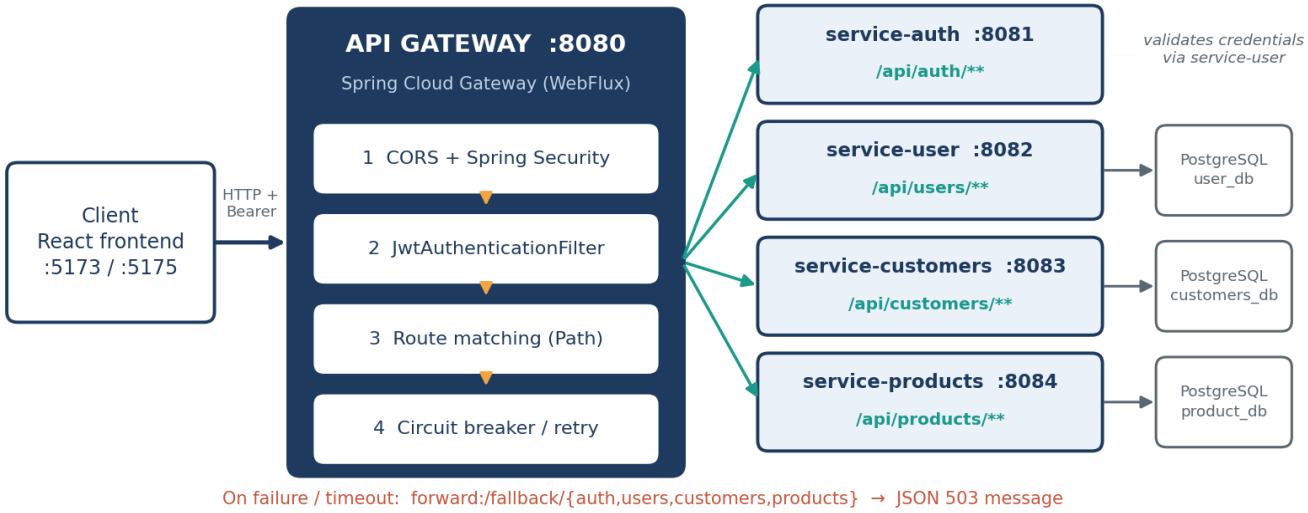


Figure 1. Request path from client through the gateway filter chain to the downstream services.

3. Route table

Path predicate	Target (default)	Circuit breaker	Fallback
`/api/auth/**`	`http://service-auth:8081`	authServiceCB (+ 3 retries on 503)	`/fallback/auth`
`/api/users/**`	`http://service-user:8082`	userServiceCB	`/fallback/users`
`/api/customers/**`	`http://service-customers:8083`	customersServiceCB	`/fallback/customers`
`/api/products/**`	`http://service-products:8084`	productsServiceCB	`/fallback/products`

Paths are forwarded unchanged (no prefix stripping), so `GET /api/products/12` on the gateway reaches `GET /api/products/12` on service-products. When a circuit opens or a call fails, the request is forwarded to the matching `/fallback/*` endpoint, which returns JSON of the form `{"status": 503, "message": "... Service is currently unavailable", "timestamp": ...}`.

4. Authentication and security

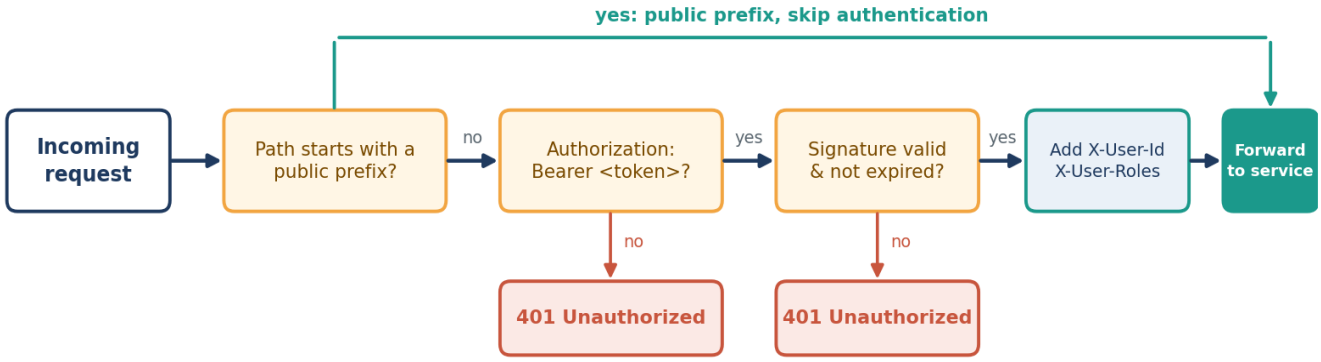


Figure 2. Decision flow of `JwtAuthenticationFilter` (a global filter, order -100).

Tokens are verified with the shared secret configured in `jwt.secret` (an HMAC key of at least 32 characters) and must not be expired. After a successful check, the gateway adds two headers that downstream services can trust and use without re-parsing the token:

Header added	Value	Source
`X-User-Id`	JWT subject (username)	`sub` claim
`X-User-Roles`	Comma-separated roles, e.g. `ADMIN,USER`	`roles` claim (empty if absent)

Public paths. Requests whose path *starts with* any of these skip JWT validation: `/api/auth/login`, `/api/auth/register`, `/api/auth/refresh-token`, `/api/products`, `/api/customers` and `/actuator/health`. Everything else, including `/api/users/**`, needs a valid `Authorization: Bearer <token>` header and otherwise gets an empty **401 Unauthorized**.

CORS. Allowed origins are `http://localhost:5173` and `http://localhost:5175` (the frontend dev servers); methods GET, POST, PUT, PATCH, DELETE, OPTIONS; credentials are allowed. CSRF protection is disabled because the API is stateless and token-based.

5. Calling the gateway

```
# 1. Log in through the gateway (public route)
curl -X POST http://localhost:8080/api/auth/login \
     -H "Content-Type: application/json" \
     -d '{"username":"alice","password":"secret"}'

# 2. Call a protected route with the returned token
curl http://localhost:8080/api/users -H "Authorization: Bearer <token>"

# 3. Check gateway health and the loaded routes
curl http://localhost:8080/actuator/health
curl http://localhost:8080/actuator/gateway/routes
```

The login request body above is illustrative; see the `service-auth` module for the exact contract.

6. Configuration and deployment

Item	Details
Port	`server.port=8080`
Config file	`api-gateway/src/main/resources/application.properties` (routes, JWT secret, CORS, Actuator)
Actuator	Exposes `health`, `info` and `gateway` endpoints
Docker	Multi-stage build (Gradle 8.10 / JDK 21, then Temurin 21 JRE Alpine); started by `docker-compose up` on `smart-sales-net`
Kubernetes	Deployment `api-gateway` (1 replica, 100m-750m CPU, 256-768 Mi) in namespace `smart-sales`; NodePort service on **30080** with TCP readiness and liveness probes
Run locally	`./gradlew :api-gateway:bootRun`, or the full stack with `docker-compose up --build`

7. Known gaps to address

Reviewing the source turned up a few items worth fixing before production use:

- **Placeholder JWT secret.** `jwt.secret` is committed in `application.properties`. It must match the secret used by service-auth and be injected from an environment variable or secret store.
- **Service URLs are hard-coded.** docker-compose sets `AUTH_SERVICE_URL`, `USER_SERVICE_URL` and similar variables, but the routes in `GatewayConfig` and `application.properties` use fixed hostnames, so the variables currently have no effect.
- **Duplicate route definitions.** The auth and user routes exist in both `application.properties` and the `GatewayConfig` Java DSL; keeping one source avoids confusion.
- **Rate limiting is not active.** `RateLimiterConfig` defines a key resolver (by `X-User-Id` or client IP) but no route applies a `RequestRateLimiter` filter.
- **Writes on customers and products are public.** Prefix matching leaves every method on these paths unauthenticated, not just reads. Restrict by HTTP method or role if that is not intended.